COMPUTER SCIENCE DEPARTMENT

Computer Science - Curriculum Artificial Intelligence

Project Assignment

Database Systems

Database Design and Implementation

Student: *Nicolò Resta*

Academic Year 2025/2026

Contents

1 Use Case Requirements **2**
1.1 Stage Up Inc. . . . 2
1.2 Common Operations . . . 2

2 Requirements Analysis **2**
2.1 Glossary of Terms . . . 4

3 Conceptual Design **5**
3.1 Business Rules . . . 5

4 Logical Design **6**
4.1 Table of Volumes . . . 6
4.2 Tables of Accesses . . . 6
4.2.1 Operation 1: Enter all data related to a new customer . . . 6
4.2.2 Operation 2: Record a new booking . . . 7
4.2.3 Operation 3: Register a new event location . . . 8
4.2.4 Operation 4: View the team that handled setups at a specific event location . . . 8
4.2.5 Operation 5: Print a list of event locations sorted by number of bookings . . . 8
4.3 ER Schema Refactoring . . . 9
4.4 Logical Schema . . . 10

5 Physical Design in Oracle **10**
5.1 Creation of types . . . 10
5.1.1 Customer Domain . . . 10
5.1.2 Geography and Infrastructure . . . 11
5.1.3 Personnel and Teams . . . 11
5.1.4 Services and Installations . . . 12
5.1.5 Bookings and Event Locations . . . 12
5.2 Creation of tables . . . 13
5.2.1 Offices, Depots, and Teams . . . 13
5.2.2 Customers . . . 13
5.2.3 Installations, Event Locations, and Bookings . . . 14
5.3 Creation of triggers . . . 14
5.3.1 Promo Validation . . . 14
5.3.2 Team Installations Counter . . . 15
5.3.3 Setup Time Estimates . . . 15
5.3.4 Employee Headcount Hierarchy Sync . . . 16
5.4 Creation of indexes . . . 17
5.5 Database Population Procedures . . . 18
5.5.1 Central Offices Population . . . 18
5.5.2 Depots Population . . . 19
5.5.3 Teams Population . . . 19
5.5.4 Installations Population . . . 20
5.5.5 Customers Population . . . 21
5.5.6 Event Locations Population . . . 22
5.5.7 Bookings Population . . . 22
5.6 Operational Procedures And Functions . . . 23
5.6.1 Customer Management . . . 23
5.6.2 Booking Management . . . 24
5.6.3 Event Location Management and Queries . . . 25

6 Web App **26**

1 Use Case Requirements

1.1 Stage Up Inc.

The company "StageUp" has a central office and several operational depots located across the world, each covering a specific region, which consists of one or more municipalities, where they manage the rental and installation of audiovisual equipment for events. The central office does not handle physical setups. Both the central office and depots are characterized by name, address, city/province, and number of employees. Bookings, either recurring or one-time, are coordinated by the central office. Bookings can be placed by customers via phone, postal mail, email, or through the website. Besides one-time bookings, the company also handles seasonal contracts and special promotional rentals. Bookings are delivered and installed at customer event locations, which are characterized by address, house number, postal code, city, and province. Since each customer can have one or more event locations, each address is identified by a unique code within the customer's profile. Additionally, each event location has a setup time estimate and equipment capacity. Depots have one or more setup teams, each with a maximum number of members. Each team is characterized by an identification code, a name, the number of installations made, and some personal data of its team members. Each booking is characterized by type, date, duration, cost, the team that handled it, and the details of the customer who placed it. Customers, classified as individuals or companies, are identified by an alphanumeric code and their personal details.

1.2 Common Operations

The most common operations on the database are the following:

1. Enter all data related to a new customer (10 times per day)
2. Record a new booking (300 times per day)
3. Register a new event location (50 times per day)
4. View the team that handled setups at a specific event location (20 times per day)
5. Print a list of event locations sorted in descending order by the number of bookings handled at each location (5 times per day)

2 Requirements Analysis

The concepts can be reorganized as follows:

Central Office
The central office does not handle physical setups
Both the central office and depots are characterized by name, address, city/province, and number of employees
Bookings, either recurring or one-time, are coordinated by the central office

Depot
Several operational depots located across the world
Each depot covers a specific region
Both the central office and depots are characterized by name, address, city/province, and number of employees
Depots have one or more setup teams, each with a maximum number of members

Region
A specific region covered by each depot
Each region consists of one or more municipalities

Booking
Bookings can be either recurring or one-time
Bookings are coordinated by the central office
Bookings can be placed by customers via phone, postal mail, email, or through the website
The company handles seasonal contracts and special promotional rentals
Bookings are delivered and installed at customer event locations
Each booking is characterized by type, date, duration, cost, the team that handled it, and the details of the customer who placed it

Customer
Customers are classified as individuals or companies
Customers are identified by an alphanumeric code and their personal details
Customers can place bookings via phone, postal mail, email, or through the website
Each customer can have one or more event locations

Event Location
Bookings are delivered and installed at customer event locations
Event locations are characterized by address, house number, postal code, city, and province
Each address is identified by a unique code within the customer's profile
Each event location has a setup time estimate and equipment capacity

Setup Team
Depots have one or more setup teams
Each team has a maximum number of members
Each team is characterized by an identification code and a name
Each team is tracked by the number of installations made
Team members have personal data stored

Equipment
Audiovisual equipment is managed for the rental and installation for events
Each event location has equipment capacity

2.1 Glossary of Terms

Concept	Description	Connections	Synonyms
Central Office	The main coordination center that does not handle physical setups but coordinates all bookings	Depot, Booking	-
Depot	Operational centers located across the world, characterized by name, address, city/province, and number of employees	Region, Setup Team	Operational Depot
Region	Geographic areas covered by depots, consisting of one or more municipalities	Depot, Municipality	-
Booking	Service requests that can be recurring or one-time, placed via various channels and characterized by type, date, duration, cost, team, and customer details	Customer, Event Location, Setup Team	-
Customer	Individuals or companies identified by alphanumeric code and personal details who place bookings and have event locations	Booking, Event Location	-
Event Location	Specific places characterized by address, house number, postal code, city, and province where bookings are delivered and installed	Customer, Booking	-
Setup Team	Groups of installation personnel working at depots, identified by code and name, tracked by number of installations and member personal data	Depot, Members	Setup Team, Installation Team, Team
Installations	Audiovisual devices managed for rental and installation at events	Booking	Audiovisual Equipment, Equipment

3 Conceptual Design

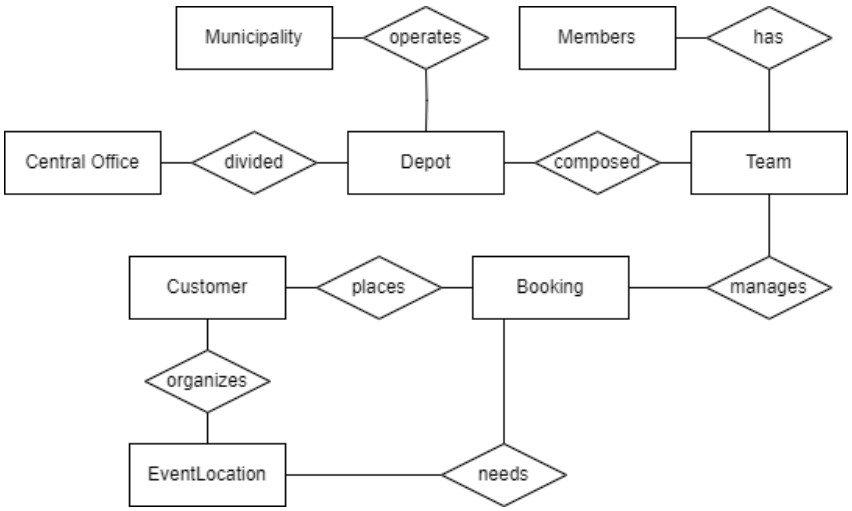


Figure 1: Conceptual skeleton schema of the database

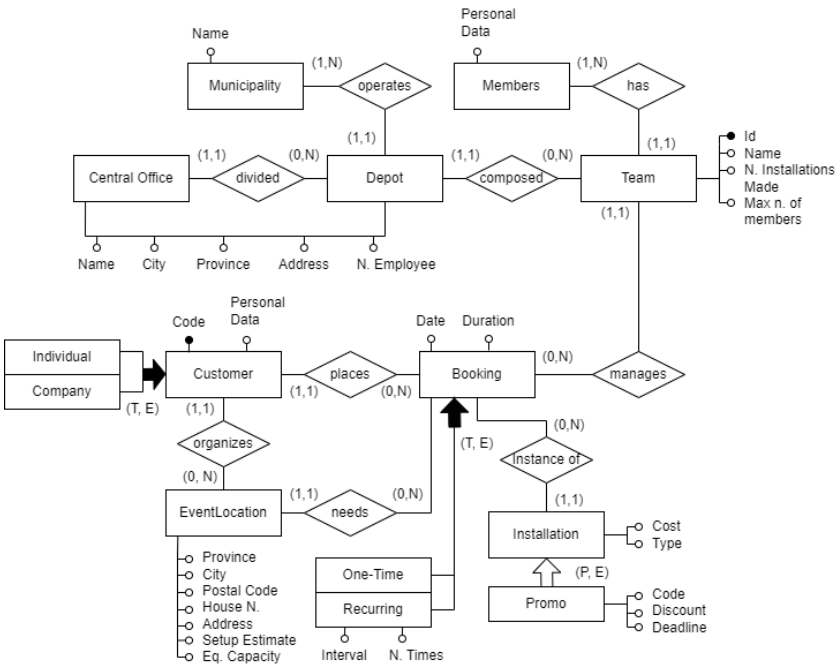


Figure 2: Conceptual ER schema of the database

3.1 Business Rules

1. The number of employees in each depot and central office is automatically calculated as the sum of the members of all teams in that depot or central office respectively.

2. The number of installations made by each team is automatically calculated as the number of bookings handled by that team.

3. Each team must have a number of members less than or equal to the maximum number of members allowed for that team.

4. Each event location setup estimate is automatically calculated as the sum of the setup times of all bookings handled at that location.

5. Each booking can be placed only by the customer who organized the event location for which the booking is made.

4 Logical Design

4.1 Table of Volumes

Concept	Concept Type	Volume	Why
Central Office	Entity	1	There is a single central office managing all operations
Divided	Relationship	10	Assuming each Central Office is divided into 10 Depots
Depot	Entity	10	Same as before
Operates	Relationship	100	By specifications each depot operates in 10 municipalities, then $10 \times 10 = 100$
Municipality	Entity	100	Same as before
Composed	Relationship	100	Assuming each depot has on average 10 teams, then $10 \times 10 = 100$
Team	Entity	100	By specifications there are 100 teams
Has	Relationship	1000	By specifications each team has on average 10 members, then $100 \times 10 = 1000$
Member	Entity	1000	Same as before
Manages	Relationship	5000	Assuming each team handles on average 50 bookings, then $100 \times 50 = 5000$
Booking	Entity	5000	Same as before
Recurrent Booking	Entity	1000	Assuming 20% of bookings are recurrent, then $5000 \times 0.2 = 1000$
One-time Booking	Entity	4000	Assuming 80% of bookings are one-time, then $5000 \times 0.8 = 4000$
Instance of	Relationship	5000	Each booking is an istance of only an installation
Installation	Entity	50	Assuming that each installation has on average 100 booking instances, then $5000/100 = 50$
Promo	Entity	10	Assuming 20% of installations are promotional, then $50 \times 0.2 = 10$
Places	Relationship	100	Assuming each customer places on average 50 bookings, then $5000/50 = 100$
Customer	Entity	100	Same as before
Individual	Entity	50	Assuming 50% of customers are individuals, then $100 \times 0.5 = 50$
Company	Entity	50	Assuming 50% of customers are companies, then $100 \times 0.5 = 50$
Organizes	Relationship	1000	Assuming each customer has on average 10 event locations, then $100 \times 10 = 1000$
Event Location	Entity	1000	Same as before
Needs	Relationship	5000	Assuming each event location has on average 5 bookings, then $1000 \times 5 = 5000$

Table 1: Estimated volumes of conceptual entities

4.2 Tables of Accesses

4.2.1 Operation 1: Enter all data related to a new customer

Concept	Concept Type	Accesses	Acc. Type	Why
Customer	Entity	1	Write	Simply insert the new customer entity with their personal details and alphanumeric code

Table 2: Accesses for Operation 1

Summary The total cost of Operation 1 is $1 \times 2 = 2$ units of cost (one write access), with a daily load cost of $2 \times 10 = 20$ units per day.

4.2.2 Operation 2: Record a new booking

For this operation, we assume the following data is provided: customer ID, event location ID, team ID, and installation ID.

Concept	Concept Type	Acc.	Acc. Type	Why
Booking	Entity	1	Write	Create a new booking entity with type, date, duration, and cost
Places	Relationship	1	Write	Establish the relationship between the booking and the customer (places)
Needs	Relationship	1	Write	Establish the relationship between the booking and the event location (needs)
Manages	Relationship	1	Write	Establish the relationship between the booking and the setup team (manages)
Instance of	Relationship	1	Write	Establish the relationship between the booking and the installation (instance of)

Table 3: Accesses for Operation 2

If we consider also the redudancy of updating the number of installations made by the team and the setup time estimate for the event location, we would have the following additional accesses:

Concept	Concept Type	Acc.	Acc. Type	Why
Team	Entity	1	Read	Read the team entity to check its current installation count
Team	Entity	1	Write	Update the number of installations made by the team (increment by 1)

If we also consider the redundancy of updating the setup time estimate for the event location, we would have the following additional accesses:

Concept	Concept Type	Acc.	Acc. Type	Why
Event Location	Entity	1	Read	Read the event location entity to check its current setup time estimate
Event Location	Entity	1	Write	Update the setup time estimate for the event location (increment by the setup time of the installation)

Summary The total cost of Operation 2 is $5 \times 2 = 10$ units of cost (five write accesses), with a daily load cost of $10 \times 300 = 3000$ units per day. If we consider also the redundancy of updating the number of installations made by the team, the total cost would be $6 \times 2 + 1 = 13$ units of cost (seven write accesses), with a daily load cost of $13 \times 300 = 3900$ units per day. If we also consider the redundancy of updating the setup time estimate for the event location, the total cost would be $6 \times 2 + 1 = 13$ units of cost (nine write accesses), with a daily load cost of $13 \times 300 = 3900$ units per day. If we consider both redundancies together, the total cost would be $7 \times 2 + 2 = 16$ units of cost (eleven write accesses), with a daily load cost of $16 \times 300 = 4800$ units per day.

4.2.3 Operation 3: Register a new event location

For this operation, we assume the customer ID is provided as input.

Concept	Concept Type	Acc.	Acc. Type	Why
Event Location	Entity	1	Write	Create a new event location entity with address, postal code, city, province, setup time estimate, and equipment capacity
Organizes	Relationship	1	Write	Establish the relationship between the event location and the customer (organizes)

Table 6: Accesses for Operation 3

Summary The total cost of Operation 3 is $2 \times 2 = 4$ units of cost (two write accesses), with a daily load cost of $4 \times 50 = 200$ units per day.

4.2.4 Operation 4: View the team that handled setups at a specific event location

For this operation, we assume the event location ID is provided as input.

Concept	Concept Type	Accesses	Acc. Type	Why
Event Location	Entity	1	Read	Access the event location to identify its associated bookings
Needs	Relationship	5	Read	Navigate the needs relationship to retrieve the bookings associated with the event location (assuming on average 5 bookings per event location)
Booking	Entity	5	Read	On average, read 5 booking entities related to the event location
Manages	Relationship	5	Read	Navigate the manages relationship to retrieve the setup teams associated with each booking
Setup Team	Entity	5	Read	Retrieve the setup team entities that handled the setups

Table 7: Accesses for Operation 4

Summary The total cost of Operation 4 is $21 \times 1 = 21$ units of cost (twenty-one read accesses), with a daily load cost of $21 \times 20 = 420$ units per day.

4.2.5 Operation 5: Print a list of event locations sorted by number of bookings

Concept	Concept Type	Accesses	Acc. Type	Why
Event Location	Entity	1000	Read	Read all event location entities to retrieve their information
Needs	Relationship	5000	Read	Navigate the needs relationship to associate bookings with event locations for counting

Table 8: Accesses for Operation 5

Summary The total cost of Operation 5 is $6000 \times 1 = 6000$ units of cost (six thousand read accesses), with a daily load cost of $6000 \times 5 = 30000$ units per day.

4.3 ER Schema Refactoring

Generalization/Specialization Analysis Individual and Company specializations of the Customer entity will hold different attributes so it is not possible to merge them together. Also the Recurring specialization of the Booking entity will hold different attributes compared to the One-time specialization so it is not possible to merge them together. Finally also the Promo specialization of the Installation entity will hold different attributes compared to the regular Installation entity so it is not possible to merge them together. Therefore, we keep the current structure of the schema with all of its generalizations or specializations.

Redudancies Analysis A first redudancy is the places relationship between Customer and Booking, which can be inferred from the organizes relationship between Customer and Event Location and the needs relationship between Event Location and Booking. Since adding this relationship would require a trigger to check that the customer placing the booking is the same as the one organizing the event location (with a non ignorable overhead), and this information can be easily inferred from the existing relationships, we decided to remove the places relationship from the schema. This decision is also supported by the fact that there are no common or frequent operations defined in the use case requirements that would require direct access to this relationship, so we can afford to infer this information when needed without incurring in significant performance issues.

The second redundancy we can identify is the number of installations made by each team, which can be calculated as the count of bookings handled by that team. Instead of storing this information as an attribute of the Team entity, we can calculate it on the fly when needed, using a query that counts the number of bookings associated with each team through the manages relationship. This would eliminate the need for redundant updates to this attribute whenever a new booking is recorded. However, since the cost of keeping this attribute updated is not significant (just one additional write access per new booking), and we could expect in the future to frequently access the number of installations made by each team, we decided to keep this attribute in the Team entity and update it whenever a new booking is recorded, accepting the redundancy for the sake of faster reads.

The third redundancy we can identify is the setup time estimate for each event location, which can be calculated as the sum of the setup times of all bookings handled at that location. Similar to the previous case, we can choose to calculate this information on the fly when needed, using a query that sums the setup times of bookings associated with each event location through the needs relationship. This would eliminate the need for redundant updates to this attribute whenever a new booking is recorded. However, also in this case the cost of keeping this attribute updated is not significant (just one additional write access per new booking), and we could expect that a customer will frequently access the setup time estimate for each event location, we decided to keep this attribute in the Event Location entity and update it whenever a new booking is recorded, accepting the redundancy for the sake of faster reads.

The fourth and fifth redudancies involve the number of employees in each depot and central office, which can be calculated as the sum of the members of all teams in that depot or central office respectively. Similar to the previous cases, we can choose to calculate this information on the fly when needed, using queries that sum the number of members of teams associated with each depot or central office through the composed relationship. This would eliminate the need for redundant updates to these attributes whenever a new team member is added or removed.

However computing such an aggregate query on the fly might be too costly in terms of performance, especially if we have a large number of teams and members. Additionally, there are no common or frequent update operations on teams, depots and central offices defined in the use case requirements that would justify the need to remove these attributes from the schema. Since updates on these involved entities are not common and we want to optimize for read performance (on those cases), we decided to keep these attributes in the Depot and Central Office entities and update them whenever a new team member is added or removed, accepting the redundancy for the sake of faster reads.

Partioning and Merging of Entities/Relationships Given the current schema, there are no candidates for partitioning or merging entities or relationships. The entities and relationships are well-defined and do not exhibit significant overlap or redundancy that would suggest a need for merging. Additionally, the entities and relationships are not large or complex enough to warrant partitioning for performance and semantical reasons. Therefore, we can keep the current structure of the schema without any partitioning or merging.

Primary Identifiers Design In order to correctly identify event locations and bookings, we added a unique numeric code automatically generated as a primary key for each new event location and booking. This will be implemented, in Oracle, using sequences that will generate a new unique value for each new record inserted into the Event Location and Booking tables. For depots, central offices and installations we choosed to use the name as primary key since it is a unique identifier for these entities and it is not expected to change over time.

4.4 Logical Schema

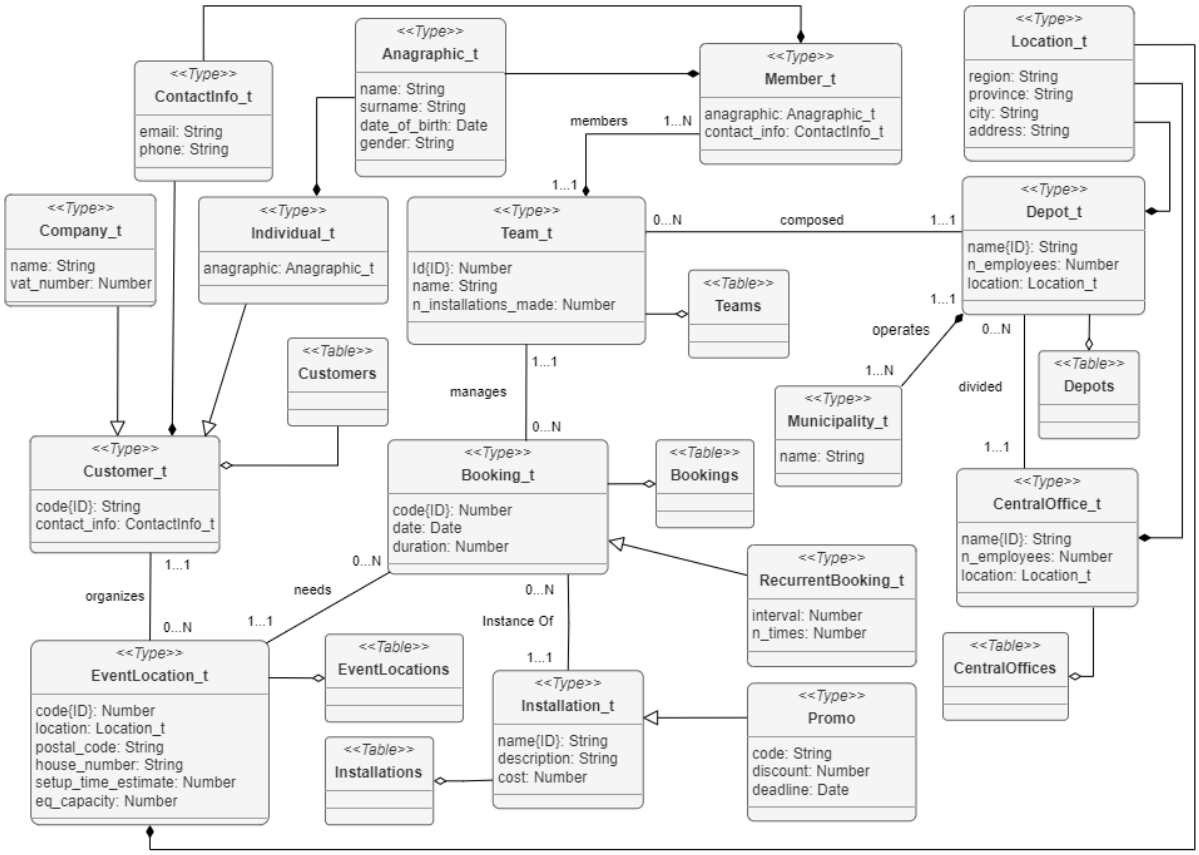


Figure 3: Logical Schema

5 Physical Design in Oracle

5.1 Creation of types

The database schema relies heavily on object types to encapsulate entities and their attributes, supporting inheritance and nested collections.

5.1.1 Customer Domain

The customer entities are modeled using inheritance, where `Individual_t` and `Company_t` inherit from the base `Customer_t` type. `ContactInfo_t` and `Anagraphic_t` keep contact and personal details structured.

```
CREATE OR REPLACE TYPE ContactInfo_t AS OBJECT (
    email VARCHAR2(100),
    phone VARCHAR2(20)
);

CREATE OR REPLACE TYPE Anagraphic_t AS OBJECT (
```

```
    name VARCHAR2(100),
    surname VARCHAR2(100),
    date_of_birth DATE,
    gender VARCHAR2(10)
);

CREATE OR REPLACE TYPE Customer_t AS OBJECT (
    code VARCHAR2(20),
    contact_info ContactInfo_t
) NOT INSTANTIABLE NOT FINAL;

CREATE OR REPLACE TYPE Individual_t UNDER Customer_t (
    anagraphic Anagraphic_t
);

CREATE OR REPLACE TYPE Company_t UNDER Customer_t (
    name VARCHAR2(100),
    vat_number VARCHAR2(50)
);
```

5.1.2 Geography and Infrastructure

Geographical properties and infrastructures such as Central Offices and Depots are modeled. Nested tables are used for municipalities covered by a depot since they represent a one-to-many relationship stricly coupled to the depot entity.

```
CREATE OR REPLACE TYPE Location_t AS OBJECT (
    region VARCHAR2(100),
    province VARCHAR2(100),
    city VARCHAR2(100),
    address VARCHAR2(200)
);

CREATE OR REPLACE TYPE CentralOffice_t AS OBJECT (
    name VARCHAR2(100),
    n_employees NUMBER,
    location Location_t
);

CREATE OR REPLACE TYPE Municipality_t AS OBJECT (
    name VARCHAR2(100)
);

CREATE OR REPLACE TYPE MunicipalityTable AS TABLE OF Municipality_t;

CREATE OR REPLACE TYPE Depot_t AS OBJECT (
    name VARCHAR2(100),
    n_employees NUMBER,
    location Location_t,
    municipalities MunicipalityTable,
    central_office REF CentralOffice_t
);
```

5.1.3 Personnel and Teams

Staff members are grouped in arrays bounded by a maximum capacity, belonging to a particular team. Since by specifications a team can have at most 10 members, a varray of 10 members is used to model the members collection.

```
CREATE OR REPLACE TYPE Member_t AS OBJECT (
    anagraphic Anagraphic_t,
    contact_info ContactInfo_t
```

```
);

CREATE OR REPLACE TYPE MembersVarray AS VARRAY(10) OF Member_t;

CREATE OR REPLACE TYPE Team_t AS OBJECT (
    id NUMBER,
    name VARCHAR2(100),
    n_installations_made NUMBER,
    depot REF Depot_t,
    members MembersVarray
);
```

5.1.4 Services and Installations

Promotions inherit from standard installations, extending them with discount information and a deadline.

```
CREATE OR REPLACE TYPE Installation_t AS OBJECT (
    name VARCHAR2(100),
    description VARCHAR2(200),
    cost NUMBER
) NOT FINAL;

CREATE OR REPLACE TYPE Promo_t UNDER Installation_t (
    code VARCHAR2(50),
    discont NUMBER,
    deadline DATE
);
```

5.1.5 Bookings and Event Locations

Event locations keep capacity and setup estimates, while bookings references participating installations, teams, and locations. Inheritance allows distinguishing recurrent bookings from one-time bookings by adding interval and repetition count attributes to the `RecurrentBooking_t` subtype. Seasonal contracts/bookings can be modeled as recurrent bookings with a specific interval and number of repetitions, allowing for flexible scheduling while maintaining the core booking structure.

```
CREATE OR REPLACE TYPE EventLocation_t AS OBJECT (
    code NUMBER,
    location Location_t,
    postal_code VARCHAR2(20),
    house_number VARCHAR2(20),
    setup_time_estimate NUMBER,
    eq_capacity NUMBER,
    customer REF Customer_t
);

CREATE OR REPLACE TYPE Booking_t AS OBJECT (
    code NUMBER,
    booking_date DATE,
    duration NUMBER,
    team REF Team_t,
    installation REF Installation_t,
    event_location REF EventLocation_t
) NOT FINAL;

CREATE OR REPLACE TYPE RecurrentBooking_t UNDER Booking_t (
    interval NUMBER,
    n_times NUMBER
) NOT FINAL;
```

5.2 Creation of tables

Object tables are instantiated from the defined object types. These tables contain constraints for subtype validation and data consistency.

5.2.1 Offices, Depots, and Teams

`CentralOffices` are connected to `Depots`, which handle nested municipal boundaries inside `MunicipalitiesNT`. Constraints check for valid `Location_t` parameters.

```
CREATE TABLE CentralOffices OF CentralOffice_t(
    name NOT NULL PRIMARY KEY,
    CONSTRAINT chk_vld_location_data_on_central_offices CHECK (
        location.province IS NOT NULL AND
        location.city IS NOT NULL AND
        location.address IS NOT NULL
    )
);

CREATE TABLE Depots OF Depot_t (
    central_office NOT NULL REFERENCES CentralOffices ON DELETE CASCADE,
    name NOT NULL PRIMARY KEY,
    CONSTRAINT chk_vld_location_data_on_depots CHECK (
        location.province IS NOT NULL AND
        location.city IS NOT NULL AND
        location.address IS NOT NULL
    )
) NESTED TABLE municipalities STORE AS MunicipalitiesNT;

CREATE TABLE Teams OF Team_t (
    id DEFAULT seq_teams.NEXTVAL NOT NULL PRIMARY KEY,
    depot NOT NULL REFERENCES Depots ON DELETE CASCADE,
    name NOT NULL
);
```

5.2.2 Customers

The `Customers` table includes heavy type discrimination logic to assert subtype attribute fulfillment (`Individual_t` versus `Company_t`) since constraints for inheritant attributes must be placed at the table level.

```
CREATE TABLE Customers OF Customer_t (
    code NOT NULL PRIMARY KEY,
    CONSTRAINT chk_contact_info CHECK (
        contact_info.email IS NOT NULL OR contact_info.phone IS NOT NULL
    ),
    CONSTRAINT chk_individual_not_null CHECK (
        TREAT(SYS_NC_ROWINFO$ AS Individual_t) IS NULL OR (
            TREAT(SYS_NC_ROWINFO$ AS Individual_t).anagraphic.name IS NOT NULL AND
            TREAT(SYS_NC_ROWINFO$ AS Individual_t).anagraphic.surname IS NOT NULL AND
            TREAT(SYS_NC_ROWINFO$ AS Individual_t).anagraphic.date_of_birth IS NOT NULL AND
            TREAT(SYS_NC_ROWINFO$ AS Individual_t).anagraphic.gender IS NOT NULL
        )
    ),
    CONSTRAINT chk_company_not_null CHECK (
        TREAT(SYS_NC_ROWINFO$ AS Company_t) IS NULL OR (
            TREAT(SYS_NC_ROWINFO$ AS Company_t).name IS NOT NULL AND
            TREAT(SYS_NC_ROWINFO$ AS Company_t).vat_number IS NOT NULL
        )
    )
);
```

5.2.3 Installations, Event Locations, and Bookings

Type casting constraints also guard constraints over `Promo_t` and `RecurrentBooking_t`.

```
CREATE TABLE Installations OF Installation_t(
    name NOT NULL PRIMARY KEY,
    description NOT NULL,
    cost NOT NULL CHECK (cost >= 0),
    CONSTRAINT chk_promo_attributes CHECK (
        TREAT(SYS_NC_ROWINFO$ AS Promo_t) IS NULL OR (
            TREAT(SYS_NC_ROWINFO$ AS Promo_t).code IS NOT NULL AND
            TREAT(SYS_NC_ROWINFO$ AS Promo_t).discont IS NOT NULL AND
            TREAT(SYS_NC_ROWINFO$ AS Promo_t).deadline IS NOT NULL AND
            TREAT(SYS_NC_ROWINFO$ AS Promo_t).discont >= 0 AND
            TREAT(SYS_NC_ROWINFO$ AS Promo_t).discont <= 100
        )
    )
);

CREATE TABLE EventLocations OF EventLocation_t (
    code DEFAULT seq_event_locations.NEXTVAL NOT NULL PRIMARY KEY,
    customer NOT NULL REFERENCES Customers ON DELETE CASCADE,
    postal_code NOT NULL,
    house_number NOT NULL,
    eq_capacity NOT NULL CHECK (eq_capacity >= 0),
    CONSTRAINT chk_vld_location_data_on_event_locations CHECK (
        location.province IS NOT NULL AND
        location.city IS NOT NULL AND
        location.address IS NOT NULL
    )
);

CREATE TABLE Bookings OF Booking_t (
    code DEFAULT seq_bookings.NEXTVAL NOT NULL PRIMARY KEY,
    team NOT NULL REFERENCES Teams ON DELETE CASCADE,
    installation NOT NULL REFERENCES Installations ON DELETE CASCADE,
    event_location NOT NULL REFERENCES EventLocations ON DELETE CASCADE,
    booking_date NOT NULL,
    duration NOT NULL CHECK (duration > 0),
    CONSTRAINT chk_recurrent_booking_attributes CHECK (
        TREAT(SYS_NC_ROWINFO$ AS RecurrentBooking_t) IS NULL OR (
            TREAT(SYS_NC_ROWINFO$ AS RecurrentBooking_t).interval IS NOT NULL AND
            TREAT(SYS_NC_ROWINFO$ AS RecurrentBooking_t).interval > 0 AND
            TREAT(SYS_NC_ROWINFO$ AS RecurrentBooking_t).n_times IS NOT NULL AND
            TREAT(SYS_NC_ROWINFO$ AS RecurrentBooking_t).n_times > 0
        )
    )
);
```

5.3 Creation of triggers

Triggers automate business logic that spans multiple entities, enforcing complex application constraints implicitly.

5.3.1 Promo Validation

A trigger on `Bookings` verifies that a tied `Promo_t` deadline has not been breached by reading it from the referenced `Installations` record.

```
CREATE OR REPLACE TRIGGER trg_validate_promo_deadline
BEFORE INSERT OR UPDATE ON Bookings
FOR EACH ROW
DECLARE
```

```
    promo_deadline DATE;
BEGIN
    IF :NEW.installation IS NOT NULL THEN
        SELECT TREAT(VALUE(i) AS Promo_t).deadline INTO promo_deadline
        FROM Installations i
        WHERE REF(i) = :NEW.installation AND VALUE(i) IS OF (Promo_t);

        IF promo_deadline IS NOT NULL AND TRUNC(SYSDATE) > promo_deadline THEN
            RAISE_APPLICATION_ERROR(-20001, 'This promo has expired and cannot be used.');
        END IF;
    END IF;
EXCEPTION
    WHEN NO_DATA_FOUND THEN NULL;
END;
```

5.3.2 Team Installations Counter

Updates the `n_installations_made` in a `Teams` record automatically upon bookings lifecycle.

```
CREATE OR REPLACE TRIGGER trg_n_installations
AFTER INSERT OR DELETE OR UPDATE ON Bookings
FOR EACH ROW
BEGIN
    IF INSERTING THEN
        UPDATE Teams t SET t.n_installations_made = NVL(t.n_installations_made, 0) + 1
        WHERE REF(t) = :NEW.team;
    ELSIF DELETING THEN
        UPDATE Teams t SET t.n_installations_made = GREATEST(NVL(t.n_installations_made, 0) -
        ↪  1, 0)
        WHERE REF(t) = :OLD.team;
    ELSIF UPDATING THEN
        IF :OLD.team != :NEW.team THEN
            UPDATE Teams t SET t.n_installations_made = GREATEST(NVL(t.n_installations_made,
            ↪  0) - 1, 0)
            WHERE REF(t) = :OLD.team;
            UPDATE Teams t SET t.n_installations_made = NVL(t.n_installations_made, 0) + 1
            WHERE REF(t) = :NEW.team;
        END IF;
    END IF;
END;
```

5.3.3 Setup Time Estimates

Maintains an aggregated setup time in `EventLocations` utilizing booking durations, with support for duration and location changes during updates.

```
CREATE OR REPLACE TRIGGER trg_setup_time
AFTER INSERT OR DELETE OR UPDATE ON Bookings
FOR EACH ROW
BEGIN
    IF INSERTING THEN
        UPDATE EventLocations e
        SET e.setup_time_estimate = NVL(e.setup_time_estimate, 0) + NVL(:NEW.duration, 0)
        WHERE REF(e) = :NEW.event_location;
    ELSIF DELETING THEN
        UPDATE EventLocations e
        SET e.setup_time_estimate = GREATEST(NVL(e.setup_time_estimate, 0) -
        ↪  NVL(:OLD.duration, 0), 0)
        WHERE REF(e) = :OLD.event_location;
    ELSIF UPDATING THEN
        IF :OLD.event_location = :NEW.event_location THEN
            UPDATE EventLocations e
```

```
            SET e.setup_time_estimate = GREATEST(NVL(e.setup_time_estimate, 0) -
            ↪  NVL(:OLD.duration, 0) + NVL(:NEW.duration, 0), 0)
            WHERE REF(e) = :NEW.event_location;
        ELSE
            UPDATE EventLocations e
            SET e.setup_time_estimate = GREATEST(NVL(e.setup_time_estimate, 0) -
            ↪  NVL(:OLD.duration, 0), 0)
            WHERE REF(e) = :OLD.event_location;
            UPDATE EventLocations e
            SET e.setup_time_estimate = NVL(e.setup_time_estimate, 0) + NVL(:NEW.duration, 0)
            WHERE REF(e) = :NEW.event_location;
        END IF;
    END IF;
END;
/
```

5.3.4 Employee Headcount Hierarchy Sync

Complex chained event propagation updates employee counts recursively upwards to Depots and Central Offices whenever Teams shift or alter their configuration natively based on the size of the members varray.

```
CREATE OR REPLACE TRIGGER trg_n_employees
AFTER INSERT OR DELETE OR UPDATE ON Teams
FOR EACH ROW
DECLARE
    v_diff NUMBER := 0;
    v_co_ref REF CentralOffice_t;
BEGIN
    IF INSERTING THEN
        v_diff := NVL(:NEW.members.COUNT, 0);
        IF v_diff != 0 AND :NEW.depot IS NOT NULL THEN
            UPDATE Depots d SET d.n_employees = NVL(d.n_employees, 0) + v_diff WHERE REF(d) =
            ↪  :NEW.depot RETURNING d.central_office INTO v_co_ref;
            IF v_co_ref IS NOT NULL THEN
                UPDATE CentralOffices c SET c.n_employees = NVL(c.n_employees, 0) + v_diff
                ↪  WHERE REF(c) = v_co_ref;
            END IF;
        END IF;
    ELSIF DELETING THEN
        v_diff := NVL(:OLD.members.COUNT, 0);
        IF v_diff != 0 AND :OLD.depot IS NOT NULL THEN
            UPDATE Depots d SET d.n_employees = GREATEST(NVL(d.n_employees, 0) - v_diff, 0)
            ↪  WHERE REF(d) = :OLD.depot RETURNING d.central_office INTO v_co_ref;
            IF v_co_ref IS NOT NULL THEN
                UPDATE CentralOffices c SET c.n_employees = GREATEST(NVL(c.n_employees, 0) -
                ↪  v_diff, 0) WHERE REF(c) = v_co_ref;
            END IF;
        END IF;
    ELSIF UPDATING THEN
        IF :OLD.depot = :NEW.depot THEN
            v_diff := NVL(:NEW.members.COUNT, 0) - NVL(:OLD.members.COUNT, 0);
            IF v_diff != 0 AND :NEW.depot IS NOT NULL THEN
                UPDATE Depots d SET d.n_employees = GREATEST(NVL(d.n_employees, 0) + v_diff,
                ↪  0) WHERE REF(d) = :NEW.depot RETURNING d.central_office INTO v_co_ref;
                IF v_co_ref IS NOT NULL THEN
                    UPDATE CentralOffices c SET c.n_employees = GREATEST(NVL(c.n_employees, 0)
                    ↪  + v_diff, 0) WHERE REF(c) = v_co_ref;
                END IF;
            END IF;
        ELSE
            -- Remove from old depot
            v_diff := NVL(:OLD.members.COUNT, 0);
            IF v_diff != 0 AND :OLD.depot IS NOT NULL THEN
```

```
                UPDATE Depots d SET d.n_employees = GREATEST(NVL(d.n_employees, 0) - v_diff,
                ↪  0) WHERE REF(d) = :OLD.depot RETURNING d.central_office INTO v_co_ref;
                IF v_co_ref IS NOT NULL THEN
                    UPDATE CentralOffices c SET c.n_employees = GREATEST(NVL(c.n_employees, 0)
                    ↪  - v_diff, 0) WHERE REF(c) = v_co_ref;
                END IF;
            END IF;
            -- Add to new depot
            v_diff := NVL(:NEW.members.COUNT, 0);
            IF v_diff != 0 AND :NEW.depot IS NOT NULL THEN
                UPDATE Depots d SET d.n_employees = NVL(d.n_employees, 0) + v_diff WHERE
                ↪  REF(d) = :NEW.depot RETURNING d.central_office INTO v_co_ref;
                IF v_co_ref IS NOT NULL THEN
                    UPDATE CentralOffices c SET c.n_employees = NVL(c.n_employees, 0) + v_diff
                    ↪  WHERE REF(c) = v_co_ref;
                END IF;
            END IF;
        END IF;
    END IF;
END;
/
```

5.4 Creation of indexes

Strategic indexes are mapped over commonly searched foreign keys to assist query optimization across large data structures.

Operations 1, 2, and 3 perform simple INSERT statements with minimal index optimization potential. Operation 1 inserts into `Customers` without complex joins, requiring no strategic indexes.

```
-----------------------------------------------------------------------------------
| Id  | Operation                | Name      | Rows  | Bytes | Cost (%CPU)| Time     |
-----------------------------------------------------------------------------------
|   0 | INSERT STATEMENT         |           |     1 |   100 |     1   (0)| 00:00:01 |
|   1 |  LOAD TABLE CONVENTIONAL | CUSTOMERS |       |       |            |          |
-----------------------------------------------------------------------------------
```

Operations 2 and 3, while referencing other tables through foreign key constraints, utilize only primary key indexes for constraint validation, which are automatically maintained by the database. These operations do not benefit from additional indexes on foreign key columns since the insertion path does not involve large-scale filtering or joins.

```
-------------------------------------------------------------------------------------------
| Id  | Operation                    | Name           | Rows  | Bytes | Cost (%CPU)| Time     |
-------------------------------------------------------------------------------------------
|   0 | INSERT STATEMENT             |                |     1 |   100 |     1   (0)| 00:00:01 |
|   1 |  LOAD TABLE CONVENTIONAL     | BOOKINGS       |       |       |            |          |
|   2 |   TABLE ACCESS BY INDEX ROWID| EVENTLOCATIONS |     1 |    35 |     1   (0)| 00:00:01 |
|*  3 |    INDEX UNIQUE SCAN         | SYS_C0010367   |     1 |       |     1   (0)| 00:00:01 |
|   4 |   TABLE ACCESS BY INDEX ROWID| INSTALLATIONS  |     1 |    74 |     1   (0)| 00:00:01 |
|*  5 |    INDEX UNIQUE SCAN         | SYS_C0010358   |     1 |       |     1   (0)| 00:00:01 |
|*  6 |   TABLE ACCESS FULL          | TEAMS          |     1 |    74 |     2   (0)| 00:00:01 |
-------------------------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   3 - access("CODE"=1)
   5 - access("NAME"='Installation 1')
   6 - filter("NAME"='Team 1')


-------------------------------------------------------------------------------------------
| Id  | Operation                    | Name           | Rows  | Bytes | Cost (%CPU)| Time     |
-------------------------------------------------------------------------------------------
|   0 | INSERT STATEMENT             |                |     1 |   100 |     1   (0)| 00:00:01 |
|   1 |  LOAD TABLE CONVENTIONAL     | EVENTLOCATIONS |       |       |            |          |
|   2 |   TABLE ACCESS BY INDEX ROWID| CUSTOMERS      |     1 |    34 |     1   (0)| 00:00:01 |
|*  3 |    INDEX UNIQUE SCAN         | SYS_C0010351   |     1 |       |     1   (0)| 00:00:01 |
-------------------------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   3 - access("CODE"='IND1')
```

Operation 4 demonstrates a scenario where index effectiveness depends on data volume. An index on `Bookings` mapped over the `event_location` foreign key reference attribute accelerates performance for retrieving location dependencies especially for the operation 4 as also its explained plan shows, where the index is used to quickly filter relevant bookings for the specified event location, reducing the search space for the join with event locations and with teams.

```
CREATE INDEX idx_bookings_event_loc ON Bookings (event_location);
```

```
-----------------------------------------------------------------------------------------------------------
| Id  | Operation                              | Name                   | Rows  | Bytes | Cost (%CPU)| Time     |
-----------------------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT                       |                        |     5 |  9700 |     5  (20)| 00:00:01 |
|   1 |  HASH UNIQUE                           |                        |     5 |  9700 |     5  (20)| 00:00:01 |
|*  2 |   HASH JOIN OUTER                      |                        |     5 |  9700 |     4   (0)| 00:00:01 |
|   3 |    NESTED LOOPS                        |                        |     5 |   275 |     1   (0)| 00:00:01 |
|   4 |     TABLE ACCESS BY INDEX ROWID        | EVENTLOCATIONS         |     1 |    35 |     1   (0)| 00:00:01 |
|*  5 |      INDEX UNIQUE SCAN                 | SYS_C0010367           |     1 |       |     1   (0)| 00:00:01 |
|   6 |     TABLE ACCESS BY INDEX ROWID BATCHED| BOOKINGS               |     5 |   100 |     0   (0)| 00:00:01 |
|*  7 |      INDEX RANGE SCAN                  | IDX_BOOKINGS_EVENT_LOC |     5 |       |     0   (0)| 00:00:01 |
|   8 |    TABLE ACCESS FULL                   | TEAMS                  |   100 |  184K|     3   (0)| 00:00:01 |
-----------------------------------------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   2 - access("P000004$"."SYS_NC_OID$"(+)="B"."SYS_NC00012$")
   5 - access("CODE"=1)
   7 - access("B"."SYS_NC00014$"="E"."SYS_NC_OID$")

Note
-----
   - dynamic statistics used: dynamic sampling (level=2)
```

Although the composite index `idx_bookings_event_loc` on the `event_location` foreign key reference is available to optimize the query, the estimated result set of only 5 rows suggests that a full table scan of `Bookings` may be equally or more efficient than index traversal and callback operations. Oracle's cost-based optimizer may therefore elect to use a table scan over the index, particularly when the indexed column selectivity is low relative to the table size, illustrating that index creation does not guarantee index usage.

```
-------------------------------------------------------------------------------------
| Id  | Operation            | Name           | Rows  | Bytes | Cost (%CPU)| Time     |
-------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT     |                |  5161 |  1789K|    28   (8)| 00:00:01 |
|   1 |  SORT ORDER BY       |                |  5161 |  1789K|    28   (8)| 00:00:01 |
|   2 |   SORT GROUP BY      |                |  5161 |  1789K|    28   (8)| 00:00:01 |
|*  3 |    HASH JOIN OUTER   |                |  5161 |  1789K|    26   (0)| 00:00:01 |
|   4 |     TABLE ACCESS FULL| BOOKINGS       |  5161 | 51610 |    19   (0)| 00:00:01 |
|   5 |     TABLE ACCESS FULL| EVENTLOCATIONS |  1001 |   337K|     7   (0)| 00:00:01 |
-------------------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   3 - access("P000003$"."SYS_NC_OID$"(+)="B"."SYS_NC00014$")

Note
-----
   - dynamic statistics used: dynamic sampling (level=2)
```

5.5 Database Population Procedures

Population procedures provide parameterized utilities to insert data into the database in a controlled manner, supporting flexible configuration and automated testing.

5.5.1 Central Offices Population

Populates the `CentralOffices` table with a specified number of entries, each assigned a location.

```
CREATE OR REPLACE PROCEDURE populate_central_offices(n IN NUMBER) IS
BEGIN
    FOR i IN 1..n LOOP
        INSERT INTO CentralOffices p (p.name, p.n_employees, p.location)
        VALUES (
```

```
            'Central Office ' || i,
            0,
            Location_t('Region' || i, 'Province' || i, 'City' || i, 'Address ' || i)
        );
    END LOOP;
    COMMIT;
END;
/
```

5.5.2 Depots Population

Creates depot records, each randomly associated with a central office and populated with municipality references.

```
CREATE OR REPLACE PROCEDURE populate_depots(n IN NUMBER) IS
    TYPE ref_co_list IS TABLE OF REF CentralOffice_t INDEX BY PLS_INTEGER;
    v_co_refs ref_co_list;
    v_co_count NUMBER;
    v_co_ref REF CentralOffice_t;
    v_num_municipalities NUMBER;
    v_municipalities MunicipalityTable;
BEGIN
    SELECT REF(c) BULK COLLECT INTO v_co_refs FROM CentralOffices c;
    v_co_count := v_co_refs.COUNT;

    FOR i IN 1..n LOOP
        v_co_ref := v_co_refs(TRUNC(DBMS_RANDOM.VALUE(1, v_co_count + 1)));
        v_num_municipalities := TRUNC(DBMS_RANDOM.VALUE(1, 6));
        v_municipalities := MunicipalityTable();
        FOR j IN 1..v_num_municipalities LOOP
            v_municipalities.EXTEND;
            v_municipalities(j) := Municipality_t('Municipality ' || i || '_' || j);
        END LOOP;

        INSERT INTO Depots (name, n_employees, location, municipalities, central_office)
        VALUES (
            'Depot ' || i,
            0,
            Location_t('Region' || (i+100), 'Province' || (i+100), 'City' || (i+100), 'Address
            ↪  ' || (i+100)),
            v_municipalities,
            v_co_ref
        );
    END LOOP;
    COMMIT;
END;
/
```

5.5.3 Teams Population

Initializes team records with randomly assigned depots and team members with generated contact information.

```
CREATE OR REPLACE PROCEDURE populate_teams(n IN NUMBER) IS
    TYPE ref_depot_list IS TABLE OF REF Depot_t INDEX BY PLS_INTEGER;
    v_depot_refs ref_depot_list;
    v_depot_count NUMBER;
    v_depot_ref REF Depot_t;
    v_num_members NUMBER;
    v_members MembersVarray;
    v_gender VARCHAR2(10);
BEGIN
```

```oracle_plsql
    SELECT REF(d) BULK COLLECT INTO v_depot_refs FROM Depots d;
    v_depot_count := v_depot_refs.COUNT;

    FOR i IN 1..n LOOP
        v_depot_ref := v_depot_refs(TRUNC(DBMS_RANDOM.VALUE(1, v_depot_count + 1)));
        v_num_members := TRUNC(DBMS_RANDOM.VALUE(1, 6));
        v_members := MembersVarray();
        FOR j IN 1..v_num_members LOOP
            v_members.EXTEND;
            IF MOD(j, 3) = 0 THEN
                v_gender := 'male';
            ELSIF MOD(j, 3) = 1 THEN
                v_gender := 'female';
            ELSE
                v_gender := 'other';
            END IF;

            v_members(j) := Member_t(
                Anagraphic_t(
                    'MName' || i || '_' || j,
                    'MSurname' || i || '_' || j,
                    TO_DATE('1985-01-01', 'YYYY-MM-DD') + MOD(i * j * 100, 10000),
                    v_gender
                ),
                ContactInfo_t(
                    'member' || i || '_' || j || '@team.com',
                    '+39' || TO_CHAR(DBMS_RANDOM.value(3000000000, 3999999999),
                    ↪  'FM0000000000')
                )
            );
        END LOOP;

        INSERT INTO Teams (id, name, n_installations_made, depot, members)
        VALUES (
            seq_teams.NEXTVAL,
            'Team ' || i,
            0,
            v_depot_ref,
            v_members
        );
    END LOOP;
    COMMIT;
END;
/
```

5.5.4 Installations Population

Populates standard installations and promotional installations with randomly generated costs and descriptions.

```oracle_plsql
CREATE OR REPLACE PROCEDURE populate_installations(n IN NUMBER, is_promo IN NUMBER) IS
BEGIN
    FOR i IN 1..n LOOP
        IF is_promo = 0 THEN
            INSERT INTO Installations VALUES (
                Installation_t(
                    'Installation ' || i,
                    'Description for installation ' || i,
                    MOD(i * 15, 1000) + 50
                )
            );
        ELSE
            INSERT INTO Installations VALUES (
```

```
                Promo_t(
                    'Promo ' || i,
                    'Description for promo ' || i,
                    MOD(i * 10, 500) + 20,
                    'CODE' || i || '_' || DBMS_RANDOM.STRING('U', 4),
                    MOD(i * 5, 50) + 5,
                    SYSDATE + MOD(i, 365)
                )
            );
        END IF;
    END LOOP;
    COMMIT;
END;
/
```

5.5.5 Customers Population

Populates both individual and company customer records with randomly generated contact details and profile attributes.

```
CREATE OR REPLACE PROCEDURE populate_customers(n IN NUMBER, is_company IN NUMBER) IS
    v_gender VARCHAR2(10);
BEGIN
    FOR i IN 1..n LOOP
        IF is_company = 0 THEN
            IF MOD(i, 2) = 0 THEN
                v_gender := 'male';
            ELSE
                v_gender := 'female';
            END IF;
            INSERT INTO Customers VALUES (
                Individual_t(
                    'IND' || i,
                    ContactInfo_t('individual' || i || '@example.com',
                                  '+39' || TO_CHAR(DBMS_RANDOM.value(3000000000, 3999999999),
                                  ↪  'FM0000000000')),
                    Anagraphic_t('IName' || i, 'ISurname' || i,
                                 TO_DATE('1970-01-01', 'YYYY-MM-DD') + MOD(i * 133, 15000),
                                 ↪  v_gender)
                )
            );
        ELSE
            INSERT INTO Customers VALUES (
                Company_t(
                    'COMP' || i,
                    ContactInfo_t('contact@company' || i || '.com',
                                  '+39' || TO_CHAR(DBMS_RANDOM.value(3000000000, 3999999999),
                                  ↪  'FM0000000000')),
                    'Company Name ' || i,
                    'IT' || TO_CHAR(DBMS_RANDOM.value(10000000000, 99999999999),
                    ↪  'FM00000000000')
                )
            );
        END IF;
    END LOOP;
    COMMIT;
END;
/
```

5.5.6 Event Locations Population

Creates event location records, each randomly associated with a customer and populated with facility details.

```
CREATE OR REPLACE PROCEDURE populate_event_locations(n IN NUMBER) IS
    TYPE ref_cust_list IS TABLE OF REF Customer_t INDEX BY PLS_INTEGER;
    v_cust_refs ref_cust_list;
    v_cust_count NUMBER;
    v_cust_ref REF Customer_t;
BEGIN
    SELECT REF(c) BULK COLLECT INTO v_cust_refs FROM Customers c;
    v_cust_count := v_cust_refs.COUNT;

    FOR i IN 1..n LOOP
        v_cust_ref := v_cust_refs(TRUNC(DBMS_RANDOM.VALUE(1, v_cust_count + 1)));

        INSERT INTO EventLocations (location, postal_code, house_number, setup_time_estimate,
        ↪  eq_capacity, customer)
        VALUES (
            Location_t('Region' || (i+200), 'Province' || (i+200), 'City' || (i+200), 'Address
            ↪  ' || (i+200)),
            TO_CHAR(MOD(i * 1234, 99999), 'FM00000'),
            TO_CHAR(MOD(i, 200) + 1),
            0,
            MOD(i, 1000) + 50,
            v_cust_ref
        );
    END LOOP;
    COMMIT;
END;
/
```

5.5.7 Bookings Population

Populates both one-time and recurrent bookings by randomly associating teams, installations, and event locations.

```
CREATE OR REPLACE PROCEDURE populate_bookings(n IN NUMBER, is_recurrent IN NUMBER) IS
    TYPE ref_team_list IS TABLE OF REF Team_t INDEX BY PLS_INTEGER;
    TYPE ref_inst_list IS TABLE OF REF Installation_t INDEX BY PLS_INTEGER;
    TYPE ref_eloc_list IS TABLE OF REF EventLocation_t INDEX BY PLS_INTEGER;
    v_team_refs ref_team_list;
    v_inst_refs ref_inst_list;
    v_eloc_refs ref_eloc_list;
    v_team_count NUMBER;
    v_inst_count NUMBER;
    v_eloc_count NUMBER;
    v_team_ref REF Team_t;
    v_inst_ref REF Installation_t;
    v_eloc_ref REF EventLocation_t;
    v_code NUMBER;
BEGIN
    SELECT REF(t) BULK COLLECT INTO v_team_refs FROM Teams t;
    v_team_count := v_team_refs.COUNT;
    SELECT REF(i) BULK COLLECT INTO v_inst_refs FROM Installations i;
    v_inst_count := v_inst_refs.COUNT;
    SELECT REF(e) BULK COLLECT INTO v_eloc_refs FROM EventLocations e;
    v_eloc_count := v_eloc_refs.COUNT;

    FOR i IN 1..n LOOP
        v_team_ref := v_team_refs(TRUNC(DBMS_RANDOM.VALUE(1, v_team_count + 1)));
        v_inst_ref := v_inst_refs(TRUNC(DBMS_RANDOM.VALUE(1, v_inst_count + 1)));
```

```
        v_eloc_ref := v_eloc_refs(TRUNC(DBMS_RANDOM.VALUE(1, v_eloc_count + 1)));

        v_code := seq_bookings.NEXTVAL;
        IF is_recurrent = 0 THEN
            INSERT INTO Bookings VALUES (
                Booking_t(
                    v_code,
                    SYSDATE + MOD(i, 30),
                    MOD(i, 8) + 1,
                    v_team_ref,
                    v_inst_ref,
                    v_eloc_ref
                )
            );
        ELSE
            INSERT INTO Bookings VALUES (
                RecurrentBooking_t(
                    v_code,
                    SYSDATE + MOD(i, 30),
                    MOD(i, 4) + 1,
                    v_team_ref,
                    v_inst_ref,
                    v_eloc_ref,
                    MOD(i, 30) + 7,
                    MOD(i, 12) + 2
                )
            );
        END IF;
    END LOOP;
    COMMIT;
END;
/
```

5.6 Operational Procedures And Functions

Operational procedures provide runtime interfaces for managing customers, bookings, and event locations at the application level, offering both data modification and query capabilities.

5.6.1 Customer Management

Dedicated procedures for adding new individual and company customers with standardized validation.

```
CREATE OR REPLACE PROCEDURE add_new_individual_customer (
    p_code          IN VARCHAR2,
    p_email         IN VARCHAR2,
    p_phone         IN VARCHAR2,
    p_name          IN VARCHAR2,
    p_surname       IN VARCHAR2,
    p_date_of_birth IN DATE,
    p_gender        IN VARCHAR2
) IS
BEGIN
    INSERT INTO Customers VALUES (
        Individual_t(
            p_code,
            ContactInfo_t(p_email, p_phone),
            Anagraphic_t(p_name, p_surname, p_date_of_birth, p_gender)
        )
    );
    COMMIT;
EXCEPTION
    WHEN OTHERS THEN
```

```
        ROLLBACK;
        RAISE_APPLICATION_ERROR(-20001, 'Error adding individual customer: ' || SQLERRM);
END add_new_individual_customer;
/

CREATE OR REPLACE PROCEDURE add_new_company_customer (
    p_code          IN VARCHAR2,
    p_email         IN VARCHAR2,
    p_phone         IN VARCHAR2,
    p_company_name  IN VARCHAR2,
    p_vat_number    IN VARCHAR2
) IS
BEGIN
    INSERT INTO Customers VALUES (
        Company_t(
            p_code,
            ContactInfo_t(p_email, p_phone),
            p_company_name,
            p_vat_number
        )
    );
    COMMIT;
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE_APPLICATION_ERROR(-20001, 'Error adding company customer: ' || SQLERRM);
END add_new_company_customer;
/
```

5.6.2 Booking Management

Procedures for managing both one-time and recurrent bookings, associating teams with installations and event locations.

```
CREATE OR REPLACE PROCEDURE add_new_one_time_booking (
    p_booking_date        IN DATE,
    p_duration            IN NUMBER,
    p_team_name           IN VARCHAR2,
    p_installation_name   IN VARCHAR2,
    p_event_location_code IN NUMBER
) IS
    v_team_ref           REF Team_t;
    v_installation_ref   REF Installation_t;
    v_event_location_ref REF EventLocation_t;
BEGIN
    SELECT REF(t) INTO v_team_ref FROM Teams t WHERE name = p_team_name;
    SELECT REF(i) INTO v_installation_ref FROM Installations i WHERE name =
    ↪  p_installation_name;
    SELECT REF(e) INTO v_event_location_ref FROM EventLocations e WHERE code =
    ↪  p_event_location_code;

    INSERT INTO Bookings VALUES (
        Booking_t(
            seq_bookings.NEXTVAL,
            p_booking_date,
            p_duration,
            v_team_ref,
            v_installation_ref,
            v_event_location_ref
        )
    );
    COMMIT;
EXCEPTION
```

```oracle
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE_APPLICATION_ERROR(-20001, 'Error adding one-time booking: ' || SQLERRM);
END add_new_one_time_booking;
/

CREATE OR REPLACE PROCEDURE add_new_recurrent_booking (
    p_booking_date        IN DATE,
    p_duration            IN NUMBER,
    p_team_name           IN VARCHAR2,
    p_installation_name   IN VARCHAR2,
    p_event_location_code IN NUMBER,
    p_interval            IN NUMBER,
    p_n_times             IN NUMBER
) IS
    v_team_ref           REF Team_t;
    v_installation_ref   REF Installation_t;
    v_event_location_ref REF EventLocation_t;
BEGIN
    SELECT REF(t) INTO v_team_ref FROM Teams t WHERE name = p_team_name;
    SELECT REF(i) INTO v_installation_ref FROM Installations i WHERE name =
    ↪  p_installation_name;
    SELECT REF(e) INTO v_event_location_ref FROM EventLocations e WHERE code =
    ↪  p_event_location_code;

    INSERT INTO Bookings VALUES (
        RecurrentBooking_t(
            seq_bookings.NEXTVAL,
            p_booking_date,
            p_duration,
            v_team_ref,
            v_installation_ref,
            v_event_location_ref,
            p_interval,
            p_n_times
        )
    );
    COMMIT;
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE_APPLICATION_ERROR(-20001, 'Error adding recurrent booking: ' || SQLERRM);
END add_new_recurrent_booking;
/
```

5.6.3 Event Location Management and Queries

Procedures for registering event locations and retrieving aggregated booking information.

```oracle
CREATE OR REPLACE PROCEDURE add_event_location (
    p_region               IN VARCHAR2,
    p_province             IN VARCHAR2,
    p_city                 IN VARCHAR2,
    p_address              IN VARCHAR2,
    p_postal_code          IN VARCHAR2,
    p_house_number         IN VARCHAR2,
    p_setup_time_estimate  IN NUMBER,
    p_eq_capacity          IN NUMBER,
    p_customer_code        IN VARCHAR2
) IS
    v_customer_ref REF Customer_t;
BEGIN
    SELECT REF(c) INTO v_customer_ref FROM Customers c WHERE code = p_customer_code;
```

```
    INSERT INTO EventLocations (code, location, postal_code, house_number,
                                setup_time_estimate, eq_capacity, customer)
    VALUES (
        seq_event_locations.NEXTVAL,
        Location_t(p_region, p_province, p_city, p_address),
        p_postal_code, p_house_number, p_setup_time_estimate, p_eq_capacity, v_customer_ref
    );
    COMMIT;
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE_APPLICATION_ERROR(-20001, 'Error registering event location: ' || SQLERRM);
END add_event_location;
/

CREATE OR REPLACE PROCEDURE get_teams_for_location (
    p_event_location_code IN NUMBER,
    p_result_cursor       OUT SYS_REFCURSOR
) IS
    v_event_location_ref REF EventLocation_t;
BEGIN
    SELECT REF(e) INTO v_event_location_ref FROM EventLocations e WHERE code =
    ↪  p_event_location_code;

    OPEN p_result_cursor FOR
        SELECT DISTINCT
            DEREF(b.team).id AS team_id,
            DEREF(b.team).name AS team_name,
            DEREF(b.team).n_installations_made AS installations
        FROM Bookings b
        WHERE b.event_location = v_event_location_ref;
END get_teams_for_location;
/

CREATE OR REPLACE PROCEDURE get_top_event_locations (
    p_result_cursor OUT SYS_REFCURSOR
) IS
BEGIN
    OPEN p_result_cursor FOR
        SELECT
            DEREF(b.event_location).code AS loc_code,
            DEREF(b.event_location).location.address AS loc_address,
            DEREF(b.event_location).location.city AS loc_city,
            COUNT(*) AS total_bookings
        FROM Bookings b
        GROUP BY b.event_location,
            DEREF(b.event_location).code,
            DEREF(b.event_location).location.address,
            DEREF(b.event_location).location.city
        ORDER BY total_bookings DESC;
END get_top_event_locations;
/
```

6 Web App

The web application was developed using **Flask**, a lightweight Python web framework, and communicates with the Oracle database using the `oracledb` library. The application provides a user-friendly dashboard with multiple forms for data entry and query operations. Each form submission triggers corresponding PL/SQL procedures stored in the database to perform the required operations. Database credentials are securely loaded from environment variables, and Jinja2 templating is used to render dynamic HTML

pages. The user interface was enhanced using **Bootstrap 5**, providing a responsive, modern design with organized card-based layouts, styled form controls, and dismissible alert messages for user feedback.

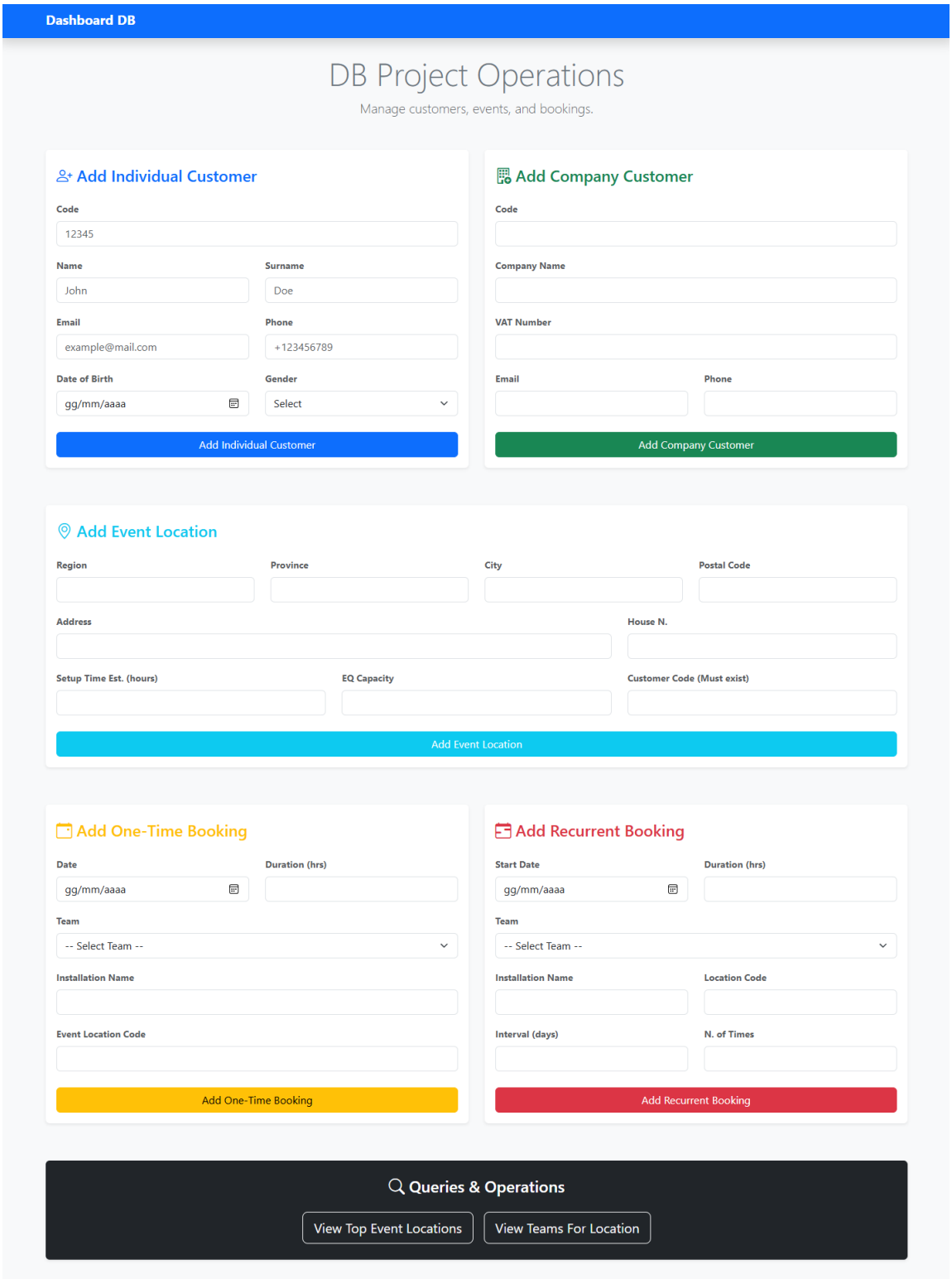


Figure 4: Screenshot of the home page of the web application

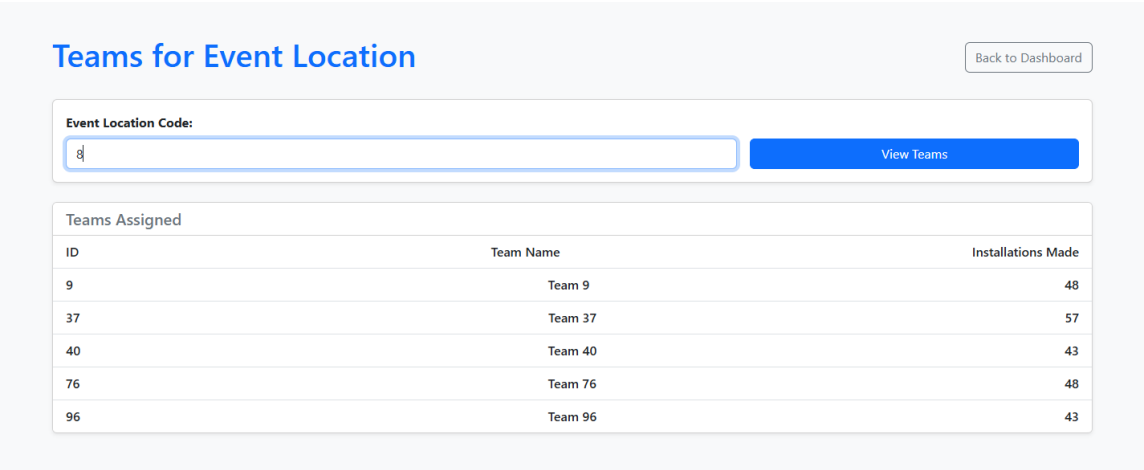
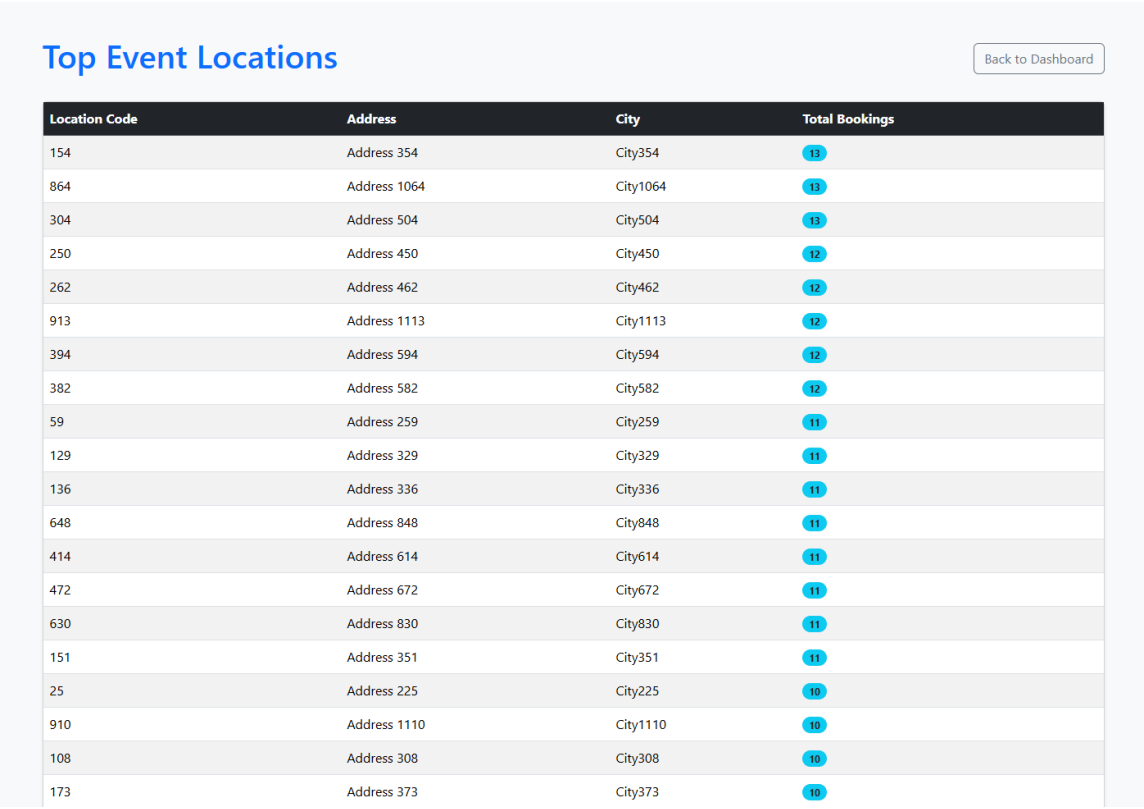


Figure 5: Screenshot of the execution of operation 4



Location Code	Address	City	Total Bookings
154	Address 354	City354	13
864	Address 1064	City1064	13
304	Address 504	City504	13
250	Address 450	City450	12
262	Address 462	City462	12
913	Address 1113	City1113	12
394	Address 594	City594	12
382	Address 582	City582	12
59	Address 259	City259	11
129	Address 329	City329	11
136	Address 336	City336	11
648	Address 848	City848	11
414	Address 614	City614	11
472	Address 672	City672	11
630	Address 830	City830	11
151	Address 351	City351	11
25	Address 225	City225	10
910	Address 1110	City1110	10
108	Address 308	City308	10
173	Address 373	City373	10

Figure 6: Screenshot of the execution of operation 5